UE : Bases de Données Relationnelles

Conception d'une base de données d'un restaurant

Étudiants :

Juba SAIB

Lyna HADJI

Lylia IMLOUL

Encadrant :

Olivier TERRAZ

Table des matières

I. Introduction 2
II. Dictionnaire des données 2
III. Dépendances Fonctionnelles 3
IV. Modélisation Conceptuelle 4
- Diagramme Entité-Association(EA) : 4
- Modèle Conceptuel des Données(MCD) 5

V. Requêtes SQL 6
VI. Declencheurs et Procédures Stockées 12
- Declencheurs 12
- Procédures Stockées 14

VII. Bibliographie 15

I. Introduction

Ce rapport présente la conception, la modélisation et la mise en œuvre d'une base de données pour un restaurant, visant à optimiser la gestion des informations sur les clients , serveurs, tables, plats, et ingrédients.

II. Dictionnaire des données

Voici le dictionnaire des données, définissant les éléments et relations clés de la base :

Table	Attributs
ingredient	nom_ingredient, unite_de_mesure, quantite_en_stock
table_restau	id_table, nbr_prs_max
plat	id_plat, nom_plat, prix
serveur	id_serveur, nom_serveur, adresse, telephone
client	id_client, nom, prenom, adresse, paye
repas	id_repas, date_repas, heure_debut, heure_fin, id_table, id_serveur, id_client
plat_ingredient	id_plat, nom_ingredient, quantite_a_utiliser
repas_plat	id_repas, id_plat, quantite
clients_impayes	id_client, nom, prenom, adresse, date_creation

III. Dépendances Fonctionnelles

Le graphe de dépendances fonctionnelles décrit les relations entre les attributs des différentes entités du système. Chaque entité est identifiée par une clé primaire qui détermine de manière unique les autres attributs associés.

- **id_serveur** détermine le nom, le téléphone et l'adresse d'un serveur
- **id_plat** détermine le nom du plat et son prix.
- **id_repas** détermine les informations liées au repas, les horaires (heure_debut, heure_fin), la date et les liens vers la table occupée (id_table), le client (id_client) et le serveur (id_serveur) .
- **nom_ingredient** identifie de manière unique les caractéristiques des ingrédients (unite_de_mesure et quantite_en_stock).

Certaines dépendances fonctionnelles lient les entités entre elles :

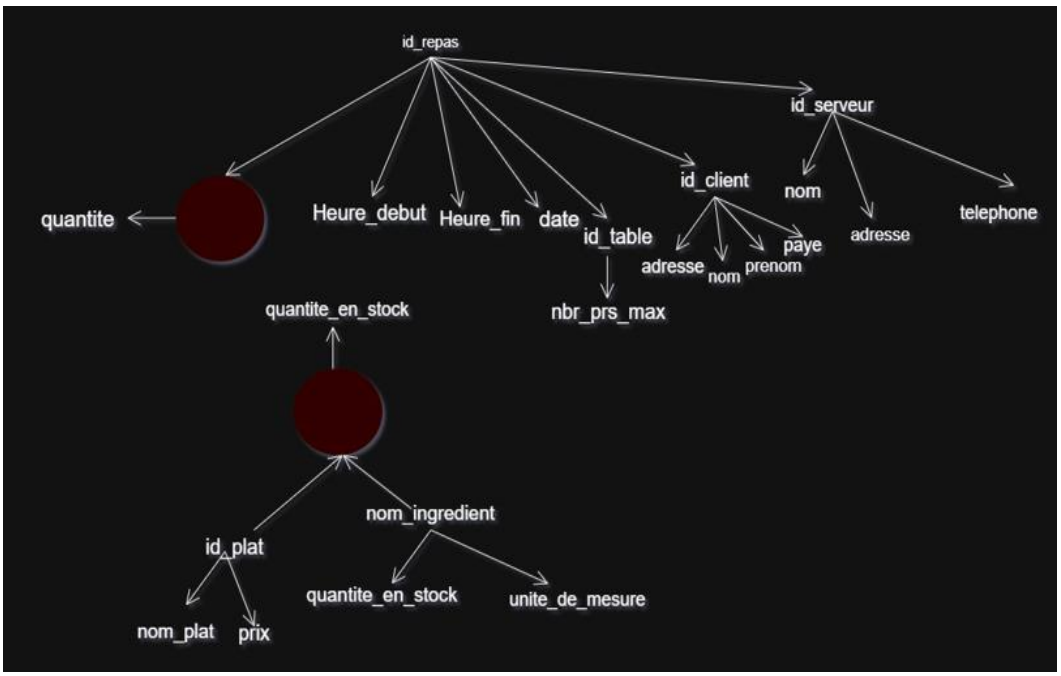
id_repas → **id_serveur** indique qu'un repas est servi par un serveur spécifique

id_repas → **id_plat** montre qu'un repas peut inclure un ou plusieurs plats via la table d'association repas_plat

id_plat → **nom_ingredient** établit le lien entre les plats et les ingrédients nécessaires en stock via la table d'association plat_ingredient.

Ces dépendances garantissent l'intégrité référentielle et la cohérence des données dans le système de gestion du restaurant.

Voici le graphe ci-dessous :



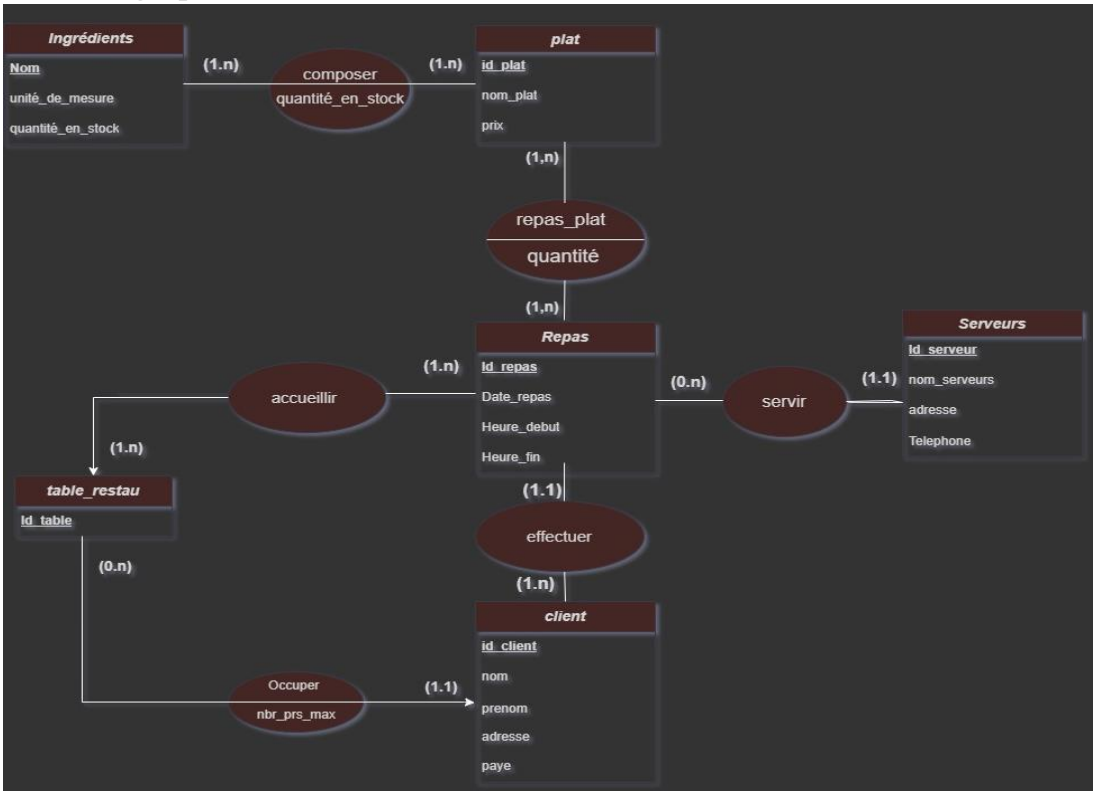
IV. Modélisation Conceptuelle

➢ Diagramme Entité-Association(EA) :

Le diagramme Entité-Association(EA) permet de visualiser graphiquement les entités, leurs attributs et les relations entre ces entités. Voici une description des principales entités et relations avec leurs cardinalités :

- **Ingredients** : Informations sur les ingrédients (nom unique, unité de mesure, quantité en stock) . Relation (1,N) avec Plat via l'association plat_ingredient, car un ingrédient peut être utilisé dans plusieurs plats et un plat contient plusieurs ingrédients.
- **Plat** : Contient les plats proposés au restaurant (id_plat, nom unique, prix). Relation(1,N) avec Repas via repas_plat, car un plat peut être consommé dans plusieurs repas et un repas peut contenir plusieurs plats.
- **Table_restau** : Informations sur les tables du restaurant(id_table, nombre de personnes maximum). Relation (1,N) avec Repas, car une table peut accueillir plusieurs repas(à des moments différents), mais un repas se déroule à une seule table.
- **Serveur** : Données des serveurs (id_serveur, nom, adresse, téléphone). Relation (1,N) avec repas, car un serveur peut servir plusieurs repas, mais chaque repas a un seul serveur responsable.
- **Client** : Informations des clients(id_client, nom, prénom,adresse, statut de paiement :paye). Relation(1,N) avec repas, car un client peut avoir consommé plusieurs repas, mais certains repas peuvent ne pas être associés à un client identifié.
- **Repas** : Enregistre les repas avec leur date, heure de début et heure de fin. Cette entité est centrale et fait le lien entre les tables, les serveurs, les clients et les plats consommés.
- **Plat_ingredient (Association)** : Relit Plat et Ingredient avec un attribut quantite_en_stock indiquant la quantité d'ingrédient nécessaire pour préparer une portion du plat. Les cardinalités sont (1,N) des deux côtés.
- **Repas_plat(Association)** : Relie repas et plat avec un attribut quantite indiquant combien de fois le plat a été commandé lors du repas. Les cardinalités sont (1,N) des deux côtés.

Voici le graphe ci-dessus :

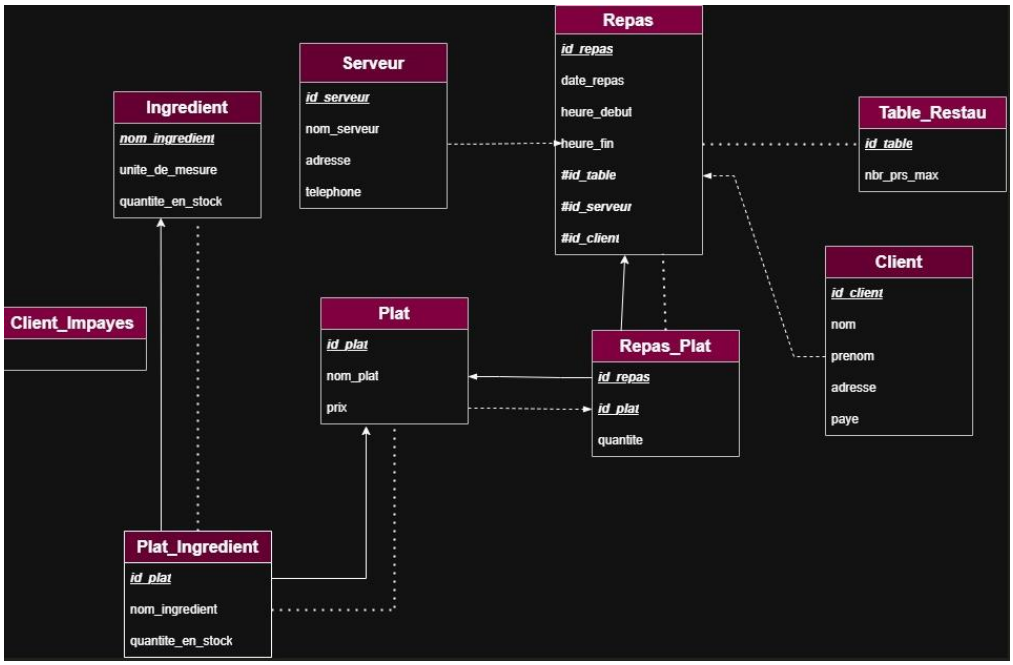


➢ Modèle Conceptuel des Données(MCD)

Le modèle Conceptuel des Données (MCD) représente les principales entités et leurs relations dans le système. Le modèle est structuré en troisième Forme Normale avec des tables définissant les entités principales et leurs relations :

- **ingredient** (**nom_ingredient**, unite_de_mesure, quantite_en_stock)
- **plat** (**id_plat**, nom_plat, prix)
- **table_restau** (**id_table**,nbr_prs_max)
- **serveur** (**id_serveur**, nom_serveur, adresse, telephone)
- **client** (**id_client**, nom, prenom, adresse, paye)
- **repas** (**id_repas**, date_repas, heure_debut, heure_fin, **#id_table**, **#id_serveur**, **#id_client**)
- **plat_ingredient** (**#id_plat**, **#nom_ingredient**, quantite_en_stock)
- **repas_plat** (**#id_repas**, **#id_plat**, quantite)

Voici le graphe ci-dessous :



V. Requêtes SQL

A partir des relations issues de la traduction en modèle relationnel, voici les requêtes SQL demandées :

1) Donner la liste des plats ayant été servis par le serveur Dupond et par le serveur Durand (les deux) :

```sql
SELECT DISTINCT plat.id_plat, plat.nom_plat FROM plat
WHERE plat.id_plat IN (
        SELECT repas_plat.id_plat FROM repas_plat
        JOIN repas  USING(id_repas)
        JOIN serveur  USING(id_serveur)
        WHERE serveur.nom_serveur = 'Dupond'
)
AND plat.id_plat IN (
        SELECT repas_plat.id_plat FROM repas_plat
        JOIN repas  USING(id_repas)
        JOIN serveur  USING(id_serveur)
        WHERE serveur.nom_serveur = 'Durand'
);
```

Résultat : La requête retourne la liste des plats concernés (id_plat, nom_plat).

id_plat	nom_plat
1	L'œuf mayonnaise
2	Terrine de volaille aux petits légumes
4	Tartare de saumon à l'aneth et citron vert
5	Salade César
10	Filet de bœuf Rossini, foie gras et truffe noire
11	Carré d'agneau rôti aux herbes, jus de romarin
12	Gambas flambées au pastis
14	Cheeseburger classique
15	Cœur de rumsteak sauce poivre
16	Carpaccio de bœuf
20	Turbot rôti au beurre blanc
21	Lotte au beurre
24	Moelleux au caramel beurre salé
25	Tarte au citron meringuée
26	Crème brulée à la pistache
27	Pain perdu au miel et amandes

Explication du fonctionnement :

1. **Jointures :** Les tables repas_plat, repas et serveur sont reliées via leurs clés communes (id_plat, id_serveur). Cela permet d'identifier quels plats ont été servis par quels serveurs.
2. **Filtrage :** Deux sous-requêtes indépendantes filtrent les plats :
   - **Sous-requête 1 :** Sélectionne les id_plat servis par le serveur « «Dupond »
   - **Sous-requête 2 :** Sélectionne les id_plat servis par le serveur « «Durand »
3. **Condition d'intersection(IN+AND)** : La clause principale utilise deux conditions IN combinés par AND. Cette requête retourne uniquement les plats présents dans les deux ensembles, c'est-à-dire les plats qui ont été servis à la fois par Dupond ET par Durand

2) **Calculer le coût du repas de la table n°23**

```
SELECT repas.id_repas, repas.id_client, client.nom, client.prenom,
ROUND(SUM(repas_plat.quantite * plat.prix), 2) AS prix_repas_client FROM repas
    JOIN repas_plat  ON repas.id_repas = repas_plat.id_repas
    JOIN plat  ON repas_plat.id_plat = plat.id_plat
    JOIN client  ON repas.id_client = client.id_client
        WHERE repas.id_table = 23
    GROUP BY repas.id_repas, repas.id_client, client.nom, client.prenom
UNION ALL
SELECT NULL AS id_repas, NULL AS id_client, 'TOTAL TABLE' AS nom, '' AS
prenom,ROUND(SUM(repas_plat.quantite * plat.prix), 2) AS prix_total_table FROM
repas
    JOIN repas_plat  ON repas.id_repas = repas_plat.id_repas
    JOIN plat  ON repas_plat.id_plat = plat.id_plat
        WHERE repas.id_table = 23
ORDER BY
    CASE WHEN id_repas IS NULL THEN 2 ELSE 1 END,
    id_repas;
```

Résultat :

id_repas	id_client	nom	prenom	prix_repas_client
25	1	SCARLETT	DAMON	32.00
26	2	JOHANSSON	MATTHEW	45.50
27	3	JOHANSSON	REY	52.00
28	4	JOHANSSON	ALBERT	32.50
		TOTAL TABLE		162.00

Explication du fonctionnement :

1. Jointures : Les tables repas, repas_plat et plat sont reliées par leurs clés communes(id_repas,id_plat) pour relier les repas aux plats commandés et aux clients.
2. Filtrage : La clause WHERE repas.id_table = 23 sélectionne uniquement les repas de la table n°23.
3. Calcul du coût total : La fonction SUM(repas_plat.quantite*plat.prix) calcule la somme des coûts en multipliant la quantité de chaque plat par son prix, puis en additionnant tous les plats du repas. La fonction ROUND arrondit le résultat à deux décimales.
4. Regroupement : La clause GROUP BY regroupe les résultats parrepas et par client(id_repas, id_client, nom, prenom) pour obtenir un total par repas individuel.

5. Combinaison avec UNION ALL : L'opérateur UNION ALL combine les résultats de deux requêtes SELECT distinctes sans éliminer les doublons. Il ajoute une ligne de total général après les lignes individuelles des clients.

6. Ligne de total général : La deuxième requête SELECT calcule le prix total de TOUS les repas de la table 23 en utilisant NULL pour id_repas et id_client, et affiche « «TOTAL TABLE » dans la colonne nom.

7. Tri des résultats : La clause ORDER BY utilise CASE WHENid_repas IS NULL THEN 2 ELSE 1 END pour placer les repas individuels en premier (valeur 1) et la ligne « TOTAL TABLE » en dernier (valeur2) , puis trie par id_repas.

3) **Donner la liste des serveurs n'ayant jamais servi le plat « lotte au beurre »**

```sql
SELECT serveur.id_serveur, serveur.nom_serveur
FROM serveur
WHERE serveur.id_serveur NOT IN (
    SELECT DISTINCT repas.id_serveur
    FROM repas
    JOIN repas_plat  ON repas.id_repas = repas_plat.id_repas
    JOIN plat  ON repas_plat.id_plat = plat.id_plat
    WHERE plat.nom_plat = 'Lotte au beurre'
);
```

Résultat : Retourne uniquement les serveurs n'ayant jamais servi ce plat spécifique. Dans notre base, les serveurs Dupond(id=1), Durand(id=2), Badana(id=7) et Maria (id=8) ont servi la lotte au beurre, tandis que les autres ne l'ont jamais servie.

id_serveur	nom_serveur
3	Dussart
6	Gatineau
7	Badana
8	Maria
9	Vaccon
10	Duguet

Explication du fonctionnement :

1. **Sous-requête :** La sous-requête interne identifie tous les serveurs(id_serveur) ayant servi au moins une fois le plat « Lotte au beurre »(id_plat =21) en joignant les tables repas, repas_plat et plat.
2. **Exclusion avec NOT IN :** La requête principale sélectionne tous les serveurs dont l'id_serveur n'apparaît pas dans le résultat de la sous-requête.

4) **Donner la liste des clients n'ayant été servis que par le serveur Dupond lors de tous les séjours effectués dans l'établissement**

```sql
SELECT client.id_client, client.nom, client.prenom,
COUNT(DISTINCT repas.id_repas) AS nombre_visites FROM client
        JOIN repas  ON client.id_client = repas.id_client
        JOIN serveur  ON repas.id_serveur = serveur.id_serveur
                GROUP BY client.id_client, client.nom, client.prenom
        HAVING COUNT(DISTINCT CASE WHEN serveur.nom_serveur = 'Dupond'
        THEN repas.id_repas END) = COUNT(DISTINCT repas.id_repas)
    AND COUNT(DISTINCT repas.id_repas) > 0
                ORDER BY client.id_client;
```

Résultat :

Retourne uniquement les clients ayant été servis exclusivement par Dupond lors de tous leurs repas dans l'établissement. Ces 4 clients n'ont eu que Dupond comme serveur pour l'ensemble de leurs visites.

id_client	nom	prenom	nombre_visites
1	SCARLETT	DAMON	3
2	JOHANSSON	MATTHEW	3
17	TOM	MIRANDA	2
18	JULIA	BARRYMORE	2

Explication du fonctionnement :

1. **Jointures :** La table client est jointe à repas via id_client, puis repas est jointe à serveur via id_serveur pour associer chaque client aux serveurs qui l'ont servi.
2. **Regroupement :** Les clients sont regroupés par leurs identifiant unique (GROUP BY client.id_client, client.nom, client.prenom) pour analyser tous leurs repas ensemble.
3. **Filtrage avec HAVING :**

- COUNT(DISTINCT CASE WHEN serveur.nom_serveur= »DUPOND » THEN repas.id_repas END) compte le nombre de repas servis par Dupond pour chaque client.
- COUNT(DISTINCT repas.id_repas) compte le nombre total de repas du client.
- La condition COUNT(DISTINCT repas.id_repas) vérifie que tous les repas du client ont été servis par Dupond.

- AND COUNT(DISTINCT repas.id_repas)>0 garantit que le client a effectivement eu aumoins un repas dans l'établissement.

5) Donner le(s) n° de la table ayant eu le coût du repas le plus élevé le 05/11/2025 entre 19h et 23h

```sql
SELECT repas.id_table, SUM(plat.prix * repas_plat.quantite) AS cout_total
FROM repas
JOIN repas_plat ON repas.id_repas = repas_plat.id_repas
JOIN plat ON repas_plat.id_plat = plat.id_plat
WHERE repas.date_repas = '2025-11-05'
AND repas.heure_debut >= '19:00:00'
AND repas.heure_fin <= '23:00:00'
GROUP BY repas.id_table
HAVING SUM(plat.prix * repas_plat.quantite) = (
    SELECT MAX(cout)
    FROM (
        SELECT SUM(p2.prix * rp2.quantite) AS cout
        FROM repas r2
        JOIN repas_plat rp2 ON r2.id_repas = rp2.id_repas
        JOIN plat p2 ON rp2.id_plat = p2.id_plat
        WHERE r2.date_repas = '2025-11-05'
        AND r2.heure_debut >= '19:00:00'
        AND r2.heure_fin <= '23:00:00'
        GROUP BY r2.id_table
    ) AS couts_tables
);
```

Résultat :

La table n°12 a généré le coût total maximum de 176.50 euros durant cette période.

id_table	cout_total
12	176.5000

Explication du fonctionnement :

1. **Filtrage temporel** : La clause WHERE filtre les repas selon la date précise (05/11/2025) entre la plage horaire (19h-23h). Cela exclut les repas en dehors de cette période.
2. **Calcul des coûts par table :** SUM (plat.prix* repas_plat.quantite) calcule le coût total pour chaque table en additionnant le prix de chaque plat multiplié par sa quantité commandée.

3. **Sous-requête pour le maximum :** La sous-requête interne calcule tous les coûts de toutes les tables pour cette période spécifique, puis MAX() identifie le coût le plus élevé parmi tous ces totaux.
4. **Clause HAVING** : Filtre pour ne retourner que les tables dont le coût total égale exactement le coût maximum, ce qui permet de gérer les cas où plusieurs tables auraient le même coût le plus élevé. Dans notre cas, seule la table 12 a ce montant maximum de 176.50 euros.

VI. Declencheurs et Procédures Stockées

➢ Declencheurs

- **Ecrire un déclencheur qui indique si un plat peut être préparé ou pas (si les ingrédients sont disponibles dans les bonnes quantités).**

```
DROP TRIGGER IF EXISTS verif_disponibilite_ingredients;
    DELIMITER $$
    CREATE DEFINER=root@localhost TRIGGER verif_disponibilite_ingredients
    BEFORE INSERT ON repas_plat FOR EACH ROW BEGIN
    IF EXISTS (
        SELECT 1 FROM plat_ingredient
        JOIN ingredient ON plat_ingredient.nom_ingredient = ingredient.nom_ingredient
        WHERE plat_ingredient.id_plat = NEW.id_plat
          AND ingredient.quantite_en_stock < plat_ingredient.quantite_a_utiliser * NEW.quantite
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stock insuffisant : impossible de préparer le plat ';
    END IF;
END$$

DELIMITER ;
```

Explication du fonctionnement :

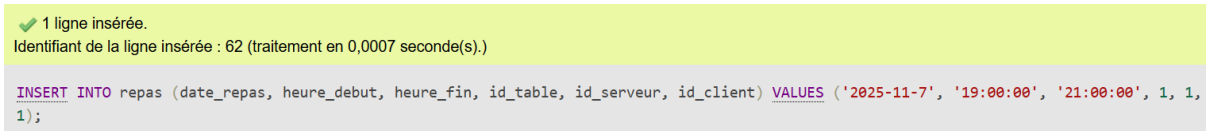
- **Avant l'insertion** : Ce trigger s'exécute avant qu'une nouvelle ligne soit insérée dans la table « repas_plat ».
- **Récupération et vérification des stocks :**A partir des tables « plat_ingredient » et « ingredient », il récupère tous les ingrédients nécessaires pour le plat (via 'NEW.id_plat') .
- **Vérification :**Si au moins un ingrédient est en quantité insuffisante('ingredients_manquants>0'), alors une erreur est levée avec un message d'erreur.

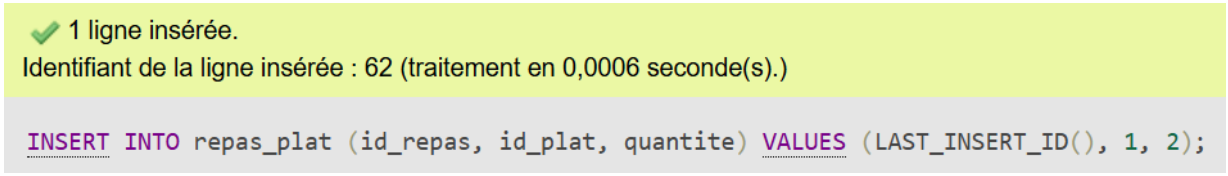
- **Mise à jour du stock :**Si tous les ingrédients sont disponibles, le trigger déduit automatiquement les quantités nécessaires du stock .

CAS 1 : Stock suffisant

```
INSERT INTO repas (date_repas, heure_debut, heure_fin, id_table, id_serveur,
id_client)
    VALUES ('2025-11-7', '19:00:00', '21:00:00', 1, 1, 1);
INSERT INTO repas_plat (id_repas, id_plat, quantite)
        VALUES (LAST_INSERT_ID(), 1, 2);
```

Résultat :

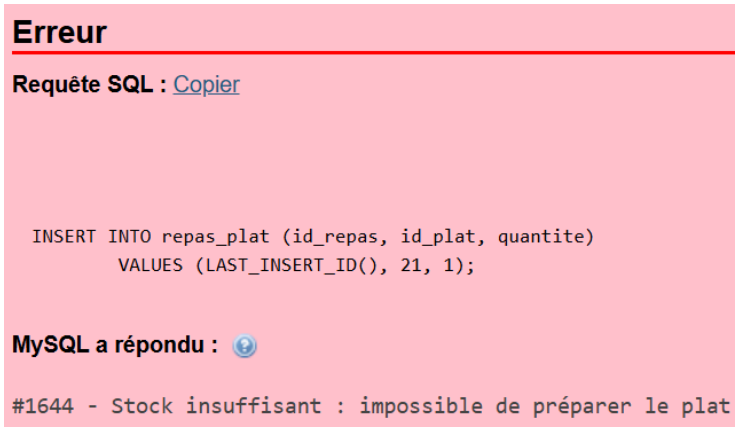




CAS 2 : Stock insuffisant

```
INSERT INTO repas (date_repas, heure_debut, heure_fin, id_table, id_serveur,
id_client)
    VALUES ('2025-11-7', '19:00:00', '21:00:00', 1, 1, 1);
INSERT INTO repas_plat (id_repas, id_plat, quantite)
    VALUES (LAST_INSERT_ID(), 21, 1);
```

Résultat :



➢ Procédures Stockées

- **Ecrire une procédure qui range dans une relation tous les clients(id,nom, prénom, adresse) n'ayant pas payé leur repas. Cette table doit être vide avant le déclanchement de la procédure**

```
      DELIMITER $$


CREATE PROCEDURE remplir_clients_impayes()
BEGIN
    -- Vider la table avant insertion
    DELETE FROM clients_impayes;

    -- Insérer les clients n'ayant pas payé
    INSERT INTO clients_impayes (id_client, nom, prenom, adresse)
    SELECT id_client, nom, prenom, adresse
    FROM client
    WHERE paye = FALSE;
END$$

DELIMITER ;
```

```
CALL remplir_clients_impayes
```

```
SELECT * FROM restaurant_bd.clients_impayes
```

Résultat :

Après l'exécution de la procédure (CALL remplir_clients_impayes), la table clients_impayes contient 5 clients impayés : JOHANSSON Albert, VACHER Jeremy, HEINICH Sufan, MOREAU Mateo etHUMPHREY Willis, avec leurs informations complètes et la date d'ajout à la liste.

id_client	nom	prenom	adresse	date_creation
4	JOHANSSON	ALBERT	30 Avenue Garibaldi, Limoges	2025-12-20 13:21:48
13	VACHER	JEREMY	15 Rue des Fleurs, Limoges	2025-12-20 13:21:48
14	HEINICH	SUFAN	22 Avenue Centrale, Limoges	2025-12-20 13:21:48
15	MOREAU	MATEO	8 Boulevard Nord, Limoges	2025-12-20 13:21:48
23	HUMPHREY	WILLIS	3 Rue de la boucherie, Limoges	2025-12-20 13:21:48

Explication du fonctionnement :

1.Création de la procédure : La commande CREATE PROCEDURE remplir_clients_impayes() définit une procédure stockée réutilisable qui peut être appelée à tout moment avec CALL remplir_clients_impayes.

2. Vidage de la table : l'instruction DELETE FROM clients_impayes supprime toutes les entrées existantes dans la table avant d'insérer les nouvelles données. Cela garantit que la table contient toujours les informations les plus récentes sans doublons.

3. Insertion des clients impayés : La commande INSERT INTO clients_impayes (id_client, nom, prenom, adresse) SELECT id_client, nom, prenom, adresse FROM client WHERE paye =FALSE copie automatiquement tous les client dont le statut de paiement est FALSE (non payé) depuis la table client vers la table clients_impayes.

4. La colonne date_creation dans la table clients_impayes est automatiquement remplie avec la date et l'heure actuelles grâce à l'attribut DEFAULT CURRENT_TIMESTAMP, permettant de tracer quand chaque client a été ajouté à la liste des impayés.

VII. Bibliographie

- Youtube : https://www.youtube.com/
- Documentation Mysql : https://dev.mysql.com/doc/
- Cours bases de données relationnelles : https://community-sciences.unilim.fr/course/view.php?id=2142